Análisis de Caso

Implementación mediante contenedores de aplicación

Eva Durán

alkemy

Análisis de Caso

Implementación mediante contenedores de aplicación

Situación inicial 📍

FinTechPlus, una empresa de servicios financieros, actualmente ejecuta su aplicación monolítica en servidores locales. Sin embargo, han surgido problemas de inconsistencias entre los entornos de desarrollo, prueba y producción, lo que dificulta el mantenimiento y la escalabilidad. Ante este desafío, la empresa ha decidido migrar la aplicación a contenedores utilizando Docker, para lograr entornos homogéneos, despliegues más rápidos y una mayor flexibilidad operativa.

Descripción del Caso 🔍

Eres un ingeniero DevOps en FinTechPlus encargado de liderar el proceso de migración de la aplicación monolítica a un entorno basado en contenedores. Tu misión consiste en:

- Analizar los desafíos actuales en la gestión de la aplicación monolítica.
- Diseñar y crear un Dockerfile que contenga todas las dependencias necesarias para ejecutar la aplicación.
- Configurar volúmenes para la persistencia de datos y establecer la conectividad de red entre el contenedor y el host.
- Documentar las ventajas y limitaciones del proceso de contenerización, y proponer recomendaciones para mejorar la escalabilidad y la eficiencia en el despliegue.

DESARROLLO

1. Descripción del caso y situación actual

FinTechPlus opera una aplicación monolítica on-premise con inconsistencias entre desarrollo, QA y producción. Para lograr entornos homogéneos, despliegues más rápidos y mayor flexibilidad, se decide contenizar la aplicación con Docker, estandarizando dependencias y empaquetado.

2. Análisis

Ventajas

- Portabilidad: la imagen incluye dependencias → se ejecuta de igual manera en diferentes entornos dev/QA/prod.
- Consistencia: el Dockerfile define un estado reproducible (capas cacheadas).
- Despliegue ágil: escalado horizontal simple.
- Aislamiento: menos conflictos de librerías con otras aplicaciones del host.
- Observabilidad: tags de imagen facilitan versionado y reversión.

Limitaciones

- Curva de aprendizaje: conceptos (capas, redes, volúmenes, registros).
- Gestión de múltiples contenedores: orquestación (Compose/ECS/K8s) añade complejidad.
- Persistencia/estado: requiere diseñar volúmenes/backups/migraciones.
- Seguridad y base de imágenes: hay que mantener imágenes base al día.

3. Implementación del Dockerfile

Ejemplo genérico para una aplicación Node monolítica (ajustar server.js/puerto según la aplicación).

```
# Imagen base ligera (Node 18 sobre Alpine Linux)
FROM node:18-alpine

# 1) Directorio de trabajo dentro de la imagen
WORKDIR /app

# 2) Variables de entorno (ajusta el puerto si tu app usa otro)
ENV NODE_ENV=production \
    PORT=3000

# 3) (Opcional) Dependencias del sistema si tu app compila módulos nativos
# RUN apk add --no-cache python3 make g++

# 4) Instala dependencias con caché eficiente:
#    Copiamos solo package*.json para aprovechar la caché de capas
COPY package*.json ./
# npm ci garantiza instalaciones reproducibles; --omit=dev para prod
RUN npm ci --omit=dev

# 5) Copiamos el resto del código fuente
COPY . .

# 6) Exponemos el puerto que atiende la app dentro del contenedor
EXPOSE 3000

# 7) (Recomendado) Ejecutar como usuario no root
# RUN addgroup -S nodegrp && adduser -S nodeusr -G nodegrp
# USER nodeusr

# 8) Comando de arranque de la aplicación
#    Si usas frameworks como Nest/Next, ajusta el entrypoint (p.ej. "node", "dist/main.js")
CMD ["node", "server.js"]
```

Build y prueba local

En terminal bash:

Construir imagen (versiona con tags)
docker build -t fintechplus/app:1.0 .

Ejecutar contenedor mapeando el puerto al host
docker run -d --name fintech-app -p 8080:3000 fintechplus/app:1.0
Navega a http://localhost:8080

4. Gestión de Volúmenes y Redes

Volúmenes (persistencia)

Docker permite configurar **volúmenes** para asegurar la persistencia de datos, separando la información del ciclo de vida efímero de los contenedores. Esto significa que aunque un contenedor se detenga, se elimine o se actualice, los datos almacenados en el volumen permanecen disponibles y pueden ser reutilizados por nuevos contenedores. Gracias a este mecanismo, es posible mantener configuraciones, archivos o bases de datos de forma segura y consistente, facilitando la portabilidad de la aplicación y evitando la pérdida de información crítica durante despliegues o actualizaciones.

Redes (comunicación entre el contenedor y el host):

Por defecto, Docker usa la red de Docker y el mapeo -p HOST:CONT expone puertos al host.

Comunicación host → contenedor: con -p 8080:3000, el host escucha en localhost:8080 y redirige al 3000 del contenedor.

Comunicación contenedor → host:

macOS/Windows: http://host.docker.internal:<puerto>.

Linux: usa la IP de docker0 o configura --add-host.

Comunicación entre contenedores en la misma red (DNS interno por nombre):

```
docker network create fintech-net
docker run -d --name api --network fintech-net fintechplus/app:1.0
docker run -d --name web --network fintech-net -p 8080:3000
fintechplus/app:1.0
# desde 'web', llamar a 'http://api:3000'
```

5. Reflexión

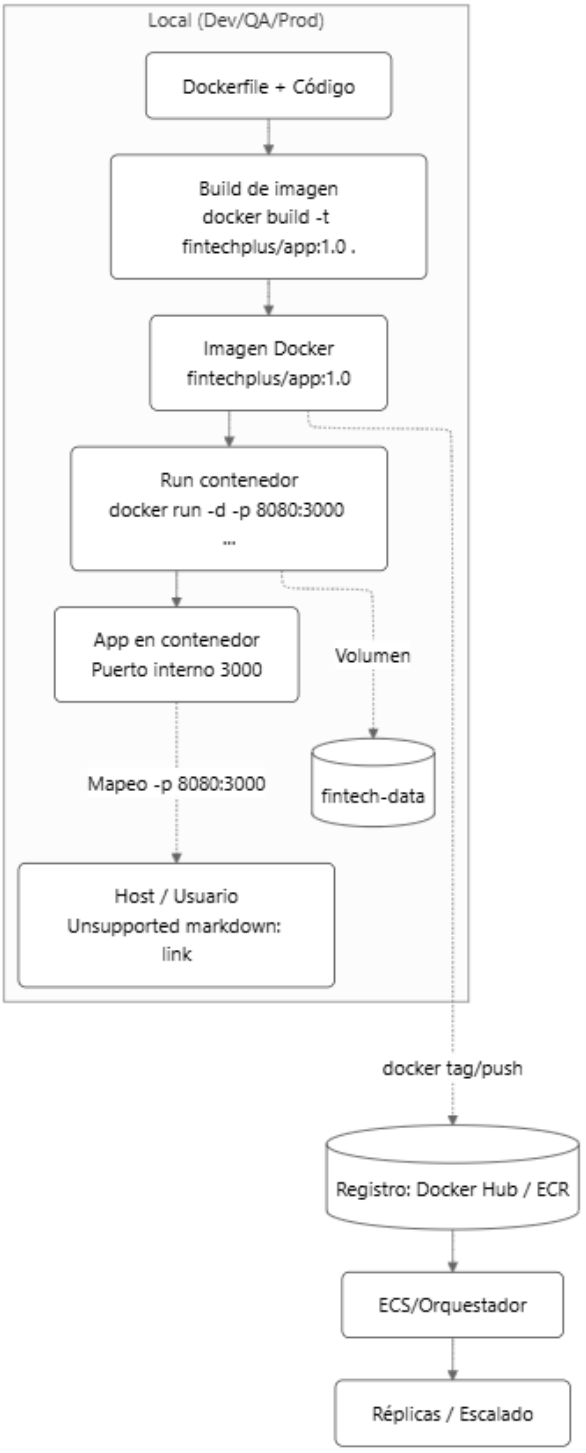
La contenerización uniformiza entornos, lo que reduce incidencias y acelera despliegues. Al empaquetar la aplicación con sus dependencias, el despliegue se vuelve predecible y repetible, habilitando escalado horizontal (más réplicas del mismo contenedor detrás de un balanceador) y observabilidad por etiquetas.

Retos: diseñar la persistencia (volúmenes, backups, migraciones) y coordinar múltiples servicios (observabilidad, healthchecks)

Recomendaciones:

- Usar imágenes slim/alpine, .dockerignore, y multi-stage builds para minimizar tamaño.
- Establecer tags inmutables (p. ej., 1.0.3, no solo latest).
- Centralizar logs/métricas (CloudWatch) y definir healthchecks.
- Si más adelante se divide el monolito, introducir Compose (local) o ECS/Kubernetes (producción) para orquestación.

6. Diagrama



El diagrama en Mermaid muestra el flujo básico de contenerización: a partir del Dockerfile y el código fuente se construye una imagen, que luego se ejecuta en un contenedor accesible desde el host mediante el mapeo de puertos y con la opción de asociar un volumen para persistir datos. Además, incluye un camino opcional en el que esa imagen puede ser enviada a un registro de contenedores (como Docker Hub o Amazon ECR) y posteriormente desplegada en AWS ECS, donde se gestionan réplicas y escalado automático para atender a los usuarios en la nube.